PopApp Buyer App — Production QA

Customer-perspective walkthrough of the live `eventtestwill` storefront, through two completed ticket orders.

TARGET	ENVIRONMENT	DATE
eventtestwill.shop.unico-app.com	Production	6 August 2026
STORE ID	**ORDERS PLACED**	**ISSUES FOUND**
367	#2670, #2671	14 (3 high)

Headline: a customer can issue themselves a valid entry pass without paying.

On the Zelle/Venmo path, clicking “Complete” is taken as proof of payment. No money moves, no verification runs — the order flips straight to *Completed*, a scannable QR pass is issued marked *Valid*, and a seat is deducted from event capacity. I confirmed this end to end on order #2670 without sending a cent.

Summary of findings

#	SEVERITY	ISSUE	AREA
1	HIGH	“Complete” self-certifies payment — unpaid order becomes Completed with a valid pass	Checkout / payment
2	HIGH	Unpaid orders consume real event inventory	Capacity
3	HIGH	Session JWT written to the browser console in plaintext	Security
4	MEDIUM	Required legal agreement is labelled `Group 1000004246.png`	Checkout
5	MEDIUM	Quantity field accepts 0 and negative values; ticket forms and price desynchronise	Checkout
6	MEDIUM	Unknown event ID renders a blank page with a live Buy Now button	Routing

#	SEVERITY	ISSUE	AREA
7	MEDIUM	Unknown route renders a permanently blank page with no way back	Routing
8	MEDIUM	Tickets “Valid” filter also lists Pending Payment tickets	Tickets
9	MEDIUM	Guest modal “Continue” drops the navigation the customer asked for	Navigation
10	MEDIUM	Loyalty points awarded for an unpaid order	Rewards
11	LOW	Third tab-bar item is a search icon leading to an empty “Custom Page 1”	Navigation
12	LOW	Guest modal renders semi-transparently over the page behind it	Visual
13	LOW	Attendee name is not remembered between orders, but email is	Checkout
14	LOW	Profile shows shipping order states (To ship, Shipped, Returns) for an events-only store	Profile

High severity

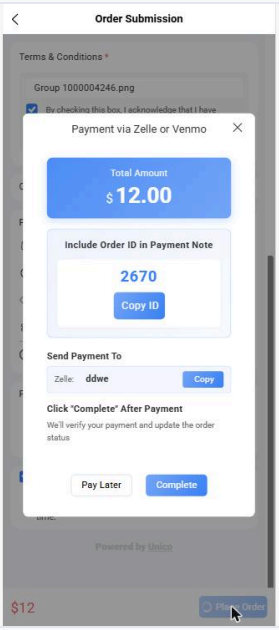
HIGH 1. “Complete” self-certifies payment

Choosing *Pay via Zelle or Venmo* opens a modal showing the amount, an order ID to quote in the payment note, and the merchant’s Zelle handle. Below it: “Click ‘Complete’ After Payment — We’ll verify your payment and update the order status.”

No verification happens. I clicked Complete without sending any money. The order immediately became **Completed**, a QR pass was issued and marked **Valid**, and it appeared under Completed in both the order list and the profile badge. Nothing distinguishes this order from a genuinely paid one.

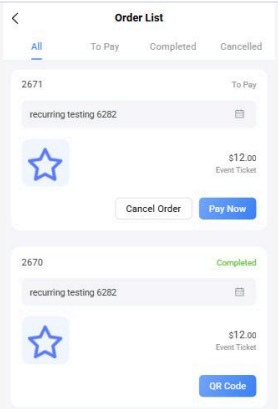
The comparison is stark: order #2671, where I clicked *Pay Later* instead, correctly sits at *Awaiting Payment* with Cancel Order / Pay Now and no pass. Both orders are equally unpaid. The only difference is which button the customer pressed.

Reproduce: pick any event → select a package → Place Order → choose Pay via Zelle or Venmo → click Complete without paying → a Valid pass is issued.



The payment modal. “Complete” is trusted without any check.

Pass 2045, marked *Valid* and ready to scan at the door — for an order that was never paid.



Both orders unpaid. #2670 (Complete) has a pass; #2671 (Pay Later) correctly does not.

HIGH

2. Unpaid orders consume real inventory

Before I ordered, the 9 August session of *recurring testing 6282* showed 50 spots left. After the unpaid order #2670, it showed 49. The seat was taken by an order that involved no payment.

Combined with issue 1, anyone can silently exhaust an event's capacity, or hold seats indefinitely, without spending anything. For a real sold-out event this blocks paying customers.

Reproduce: note the "spots left" on a session, place an order, return to the event page.

HIGH

3. Session token logged to the browser console

On every page load the app writes the signed session JWT to the browser console in plaintext, prefixed `user.token`. The payload decodes to the store ID, user ID, user type, subscription tier and expiry.

Anything with console access — a browser extension, a support agent asked to "open developer tools", a screen recording, a bug-report tool that captures console output — captures a working session credential. The token I observed carries a very long expiry.

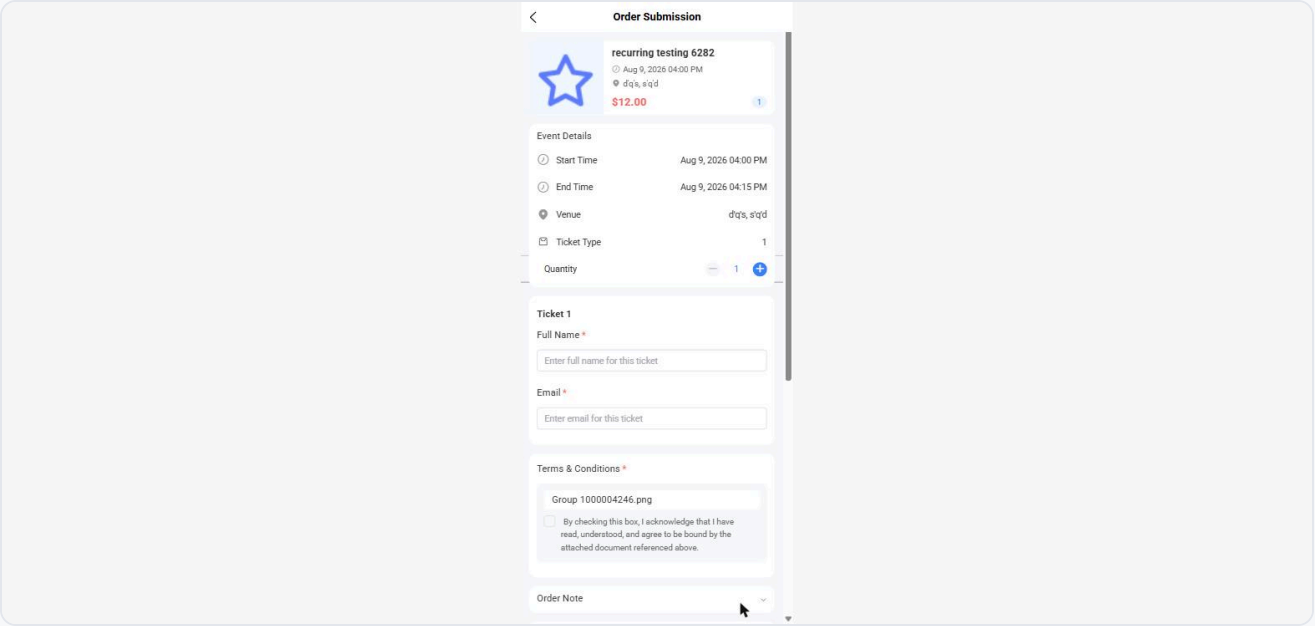
Reproduce: open the storefront with DevTools console open and filter for `user.token`. Fix: strip the log before production builds.

Medium severity

MEDIUM **4. Legal agreement labelled with a raw filename**

The mandatory Terms & Conditions block presents the document a customer must agree to be “bound by” as `Group 1000004246.png`. That is an export artefact name from a design tool, not a document title. Customers are asked to accept a binding agreement they cannot identify.

This is the same defect seen on the dev store, so it is not store-specific configuration.

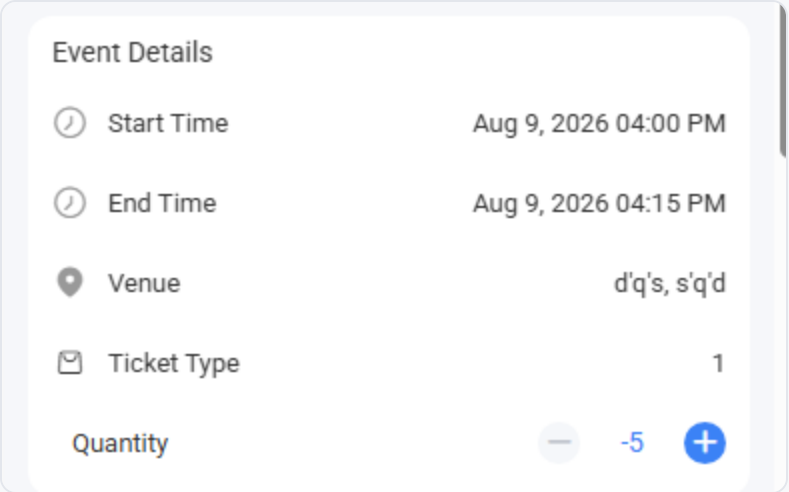


The document the customer must agree to, named after a design-tool export.

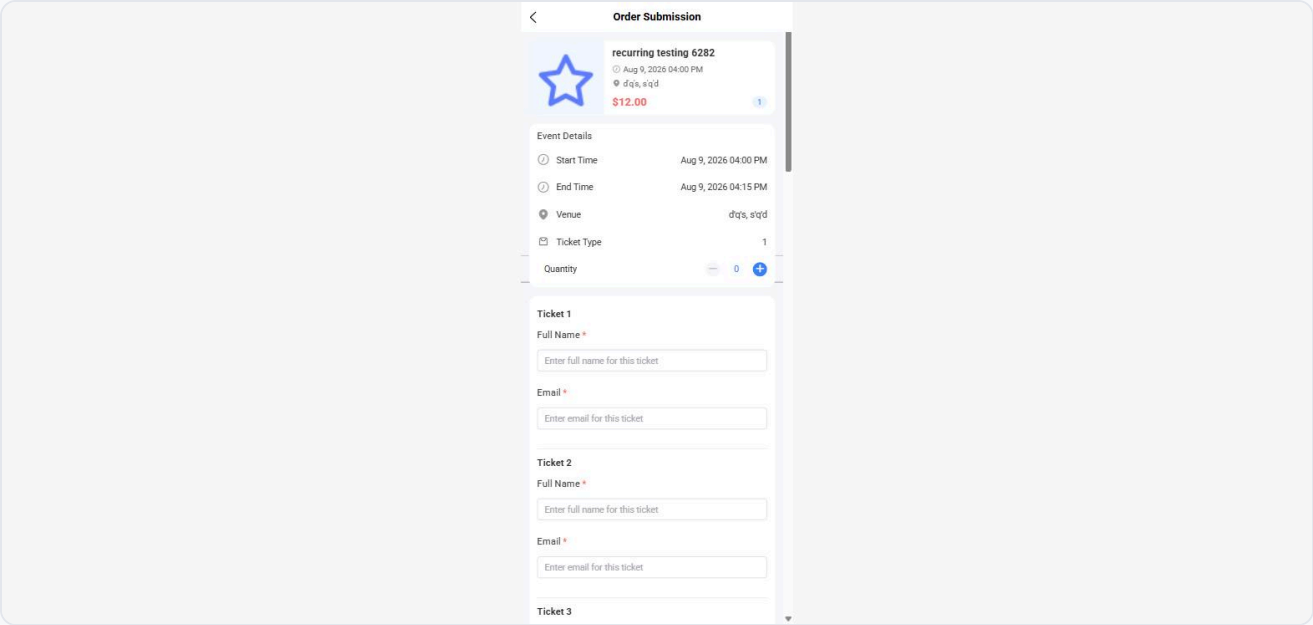
MEDIUM **5. Quantity accepts 0 and negative values**

The quantity stepper is capped at 10 via its buttons, but the underlying field accepts any typed value. Setting it to 0 leaves ten attendee forms rendered and the price stuck at the previous total. Setting it to −5 displays “−5” next to the stepper while the price still reads the old figure.

The order cannot actually be submitted in this state — server-side validation holds — but the customer sees a total that does not match what they selected, which erodes trust at the moment of payment.



Quantity showing −5.



Quantity 0, ten attendee forms still rendered, price unchanged.

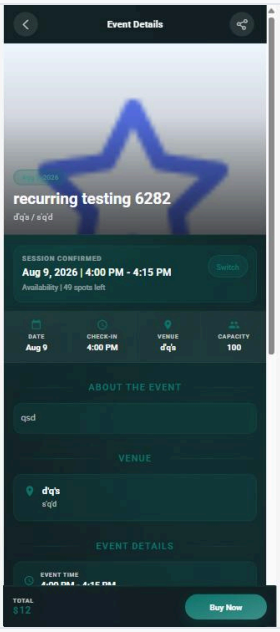
MEDIUM

6. Unknown event ID gives a blank page with a live Buy button

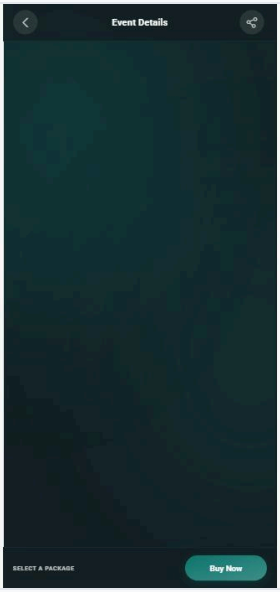
Opening `#/pages/event/detail?id=999999` loads an Event Details page that is entirely empty except for the header and a working “Buy Now” button. There is no “event not found” message. Pressing Buy Now does nothing at all — no error, no toast.

Worse, if the customer navigates there from another event rather than loading it fresh, the page shows the *previous* event’s title, venue, date, capacity and price, with a live Buy button.

A stale or mistyped link therefore shows a customer an event that is not the one they requested.



Event ID 999999 showing the previously viewed event's details.



The same URL loaded fresh: empty, with Buy Now still offered.

MEDIUM

7. Unknown route renders a dead blank page

Any hash route that does not exist — for example `#/pages/user/index` — renders a completely white page with no header, no message and no navigation. The tab title keeps the previous page's name. The customer's only escape is the browser back button; there is nothing on screen to click.

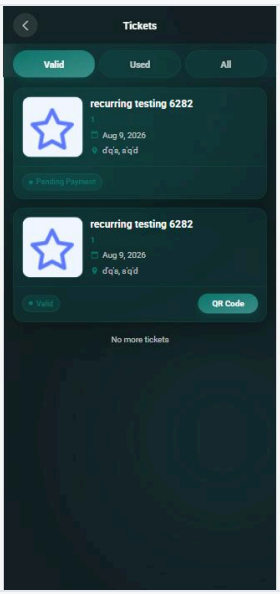
An unknown route: no content, no error, no way out.

MEDIUM

8. “Valid” ticket filter shows Pending Payment tickets

The Tickets page offers Valid / Used / All. The Valid tab returns a ticket badged *Pending Payment* alongside the genuinely valid one — its contents are byte-for-byte identical to the All tab. The filter is not filtering.

A customer glancing at “Valid” reasonably concludes they are ready to attend, when in fact one ticket is unpaid and will presumably be refused at the door.

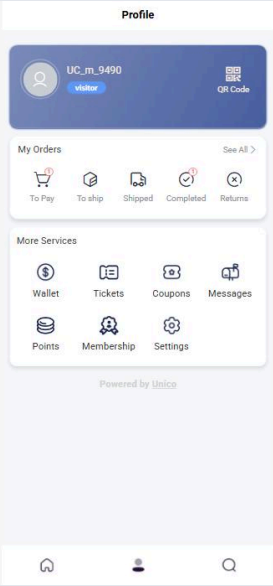


Under “Valid”: one Pending Payment ticket and one actually valid one.

MEDIUM 9. Guest modal “Continue” drops the requested navigation

Tapping Points (or another gated item) as a guest raises a “Friendly Reminder” offering Continue or Sign In Now. Choosing Continue — the option that means “carry on as a guest” — dismisses the modal and leaves the customer on the Profile page. The destination they tapped is never opened.

The result is a menu item that appears simply not to work. This also reproduced on the dev store.



After Continue: still on Profile, Points never opened.

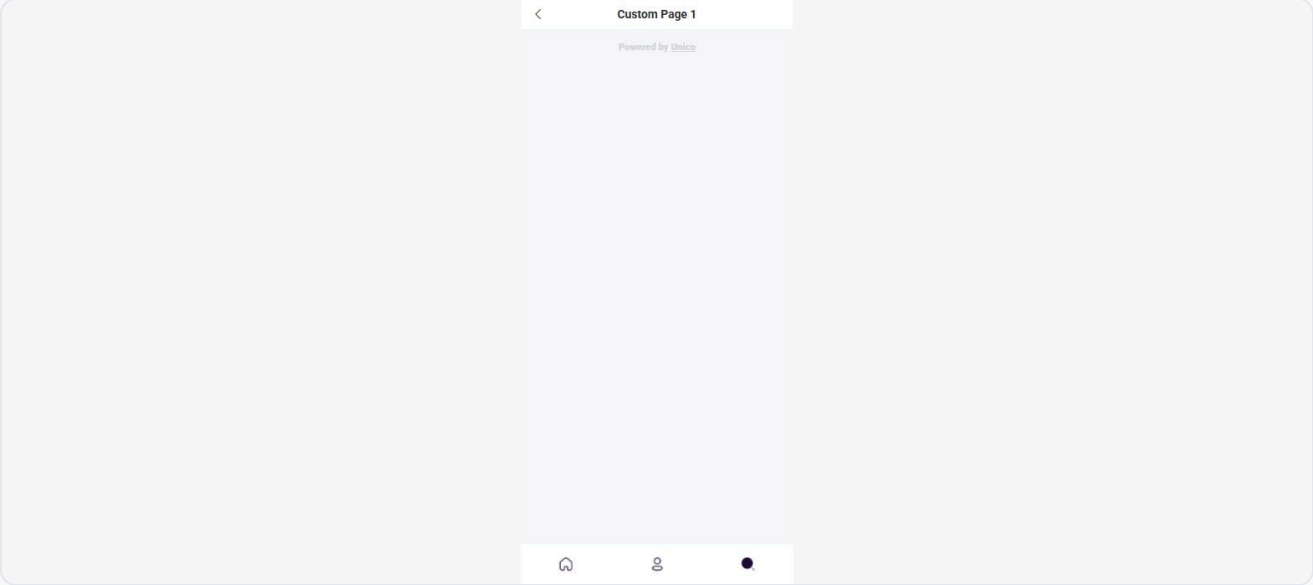
MEDIUM 10. Loyalty points granted for an unpaid order

The checkout price panel read “You have 0 points” before my first order. After the unpaid order #2670 completed, it read “You have 108 points”. Points are therefore awarded on the self-certified completion described in issue 1, not on payment — so the same loophole also mints redeemable loyalty currency.

Low severity

LOW 11. Third tab leads to an empty “Custom Page 1”

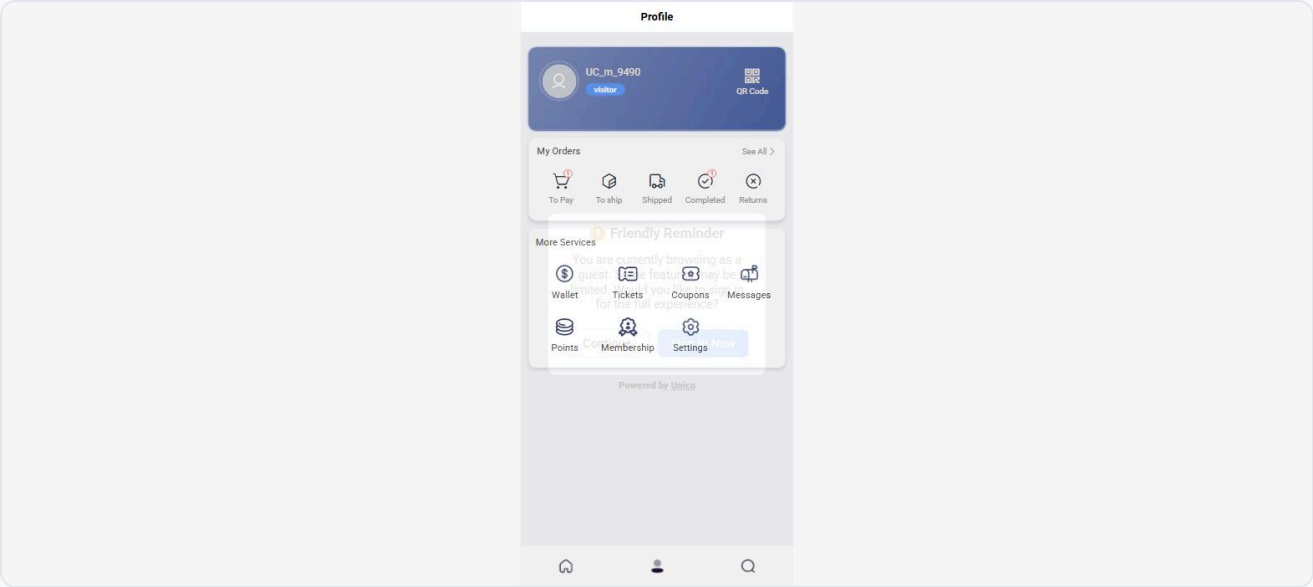
The tab bar's third item uses a magnifying-glass icon, which customers will read as search. It opens a page titled "Custom Page 1" containing nothing but the "Powered by Unico" footer. Its active-state icon also renders as a solid black blob rather than the magnifier.



"Custom Page 1" — reachable from the main tab bar, entirely empty.

LOW **12. Guest modal renders semi-transparently**

The "Friendly Reminder" modal is drawn with a partly transparent surface, so the menu grid behind it shows through and the two layers of text overlap. It is hard to read and looks broken.



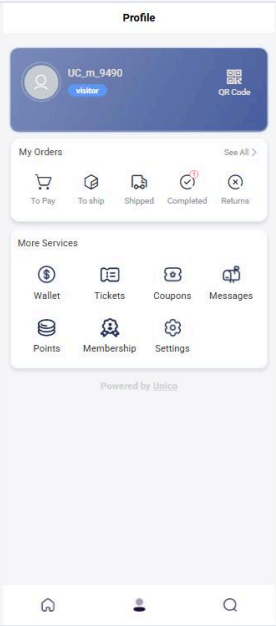
Modal text and the menu underneath rendering on top of each other.

LOW 13. Attendee name is forgotten, email is remembered

Starting a second order pre-fills the email from the previous purchase but leaves Full Name empty. Remembering half of a two-field pair is more confusing than remembering neither — the form looks partly complete, so the empty name is easy to miss until validation rejects it.

LOW 14. Shipping order states on an events-only store

The profile's My Orders row shows To Pay, To ship, Shipped, Completed and Returns. For a store selling only event tickets, three of those five will always be empty and imply a fulfilment process that does not exist.



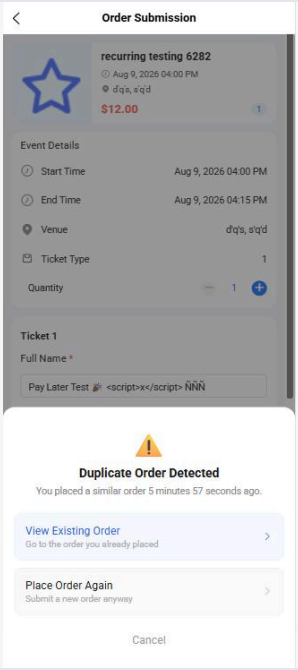
Shipping states on an events-only storefront.

What worked well

Several things are clearly better here than on the dev store, and are worth recording so they are not regressed:

- GOOD **Validation is present and specific.** Place Order produces clear toasts: "Ticket 1: please enter full name", "Ticket 1: please enter a valid email address", "Please agree to all terms and conditions before proceeding." The silent-failure behaviour seen on dev does not occur here.

- GOOD **Duplicate-order guard.** Re-submitting a similar order raises “Duplicate Order Detected — you placed a similar order 5 minutes 57 seconds ago”, offering View Existing Order or Place Order Again. A genuinely useful safeguard against double-charging.
- GOOD **Input is handled safely.** A name containing emoji, accented characters and a literal `<script>` tag was stored and redisplayed as inert text on the order and pass. Whitespace-only names are rejected.
- GOOD **Pay Later is correct.** It produces an honest “Awaiting Payment” order with Cancel Order and Pay Now, and correctly withholds the pass.
- GOOD **The recurring-series picker is clear.** Choosing among seven sessions, confirming, and switching sessions all behaved sensibly, with per-session availability shown.
- GOOD **The confirmation page renders immediately.** The indefinite “verifying your payment” hang seen on dev did not occur.



The duplicate-order guard.

Suggested priority

1. **Stop treating “Complete” as proof of payment.** The customer’s click should move the order to a pending-verification state that a merchant confirms, not to Completed. Until then, no pass should be issued and no points granted.
2. **Do not deduct inventory for unverified orders**, or hold the seat on a timer that releases it.
3. **Remove the `user.token` console log** from production builds.

4. **Give the Terms & Conditions document a real title.** This is a one-line fix on a legally binding agreement.
5. **Fix the Valid ticket filter** so unpaid tickets are excluded.
6. **Add a not-found state** for unknown routes and unknown event IDs, and clear stale event data before loading a new one.
7. **Make the guest modal's Continue complete the navigation**, and give it an opaque background.
8. Clamp the quantity field server-side and client-side; hide shipping order states for events-only stores; either populate or remove Custom Page 1.

Scope. Testing was performed as an anonymous guest customer against the live production storefront on 6 August 2026, using the Chrome browser at desktop width. Two orders were placed on the `recurring testing 6282` event: **#2670** (Completed, unpaid — the subject of issue 1) and **#2671** (Awaiting Payment). Both remain on the store and should be cancelled.

Not covered: the Online Payment / card path — no card or bank details were entered at any point; refund and cancellation flows; coupon redemption; points redemption; merchant-side check-in scanning; and any non-H5 target (iOS, Android, mini-program). Payloads were limited to benign input; no injection or load testing was performed against production.